

• 对于访问与修改最坏 $O(1)$ 时间的讨论

朴素的思路, 从高层到低层扫描, 看最后插入的在哪一层, 这样的做法复杂度为 $O(r)$

如何压缩? 使用广义 B 进制!

设 n_0 : 最后一个部分 B-size block 的元素个数

n_1 : full B-size block 数量

n_2 : ---

n_{r-1} : full B^{r-1} -size blocks 数量

$$\star n < 2B$$

考虑这样一个过程. 我们有一个 n 进制数 x ,

有 k 个元素相当于加了 k 次 1. 而这整个加法过程是确定的, 那么对于这一过程中的任意一个 k , 加它完时候, 必有最低位 (非零) 是它放入的块!

因此

☆☆ 能上升 至少 一层, 且不会继续上升

这是一个核心观点, 如何上升? 有新元素加入!

设 N_k 表示低于 level k 的那些块中的元素总数

所以直观说, 一个元素在哪一层, 是由后面有多少元素决定的

一个很重要的 是: 对于所有在某元素后面

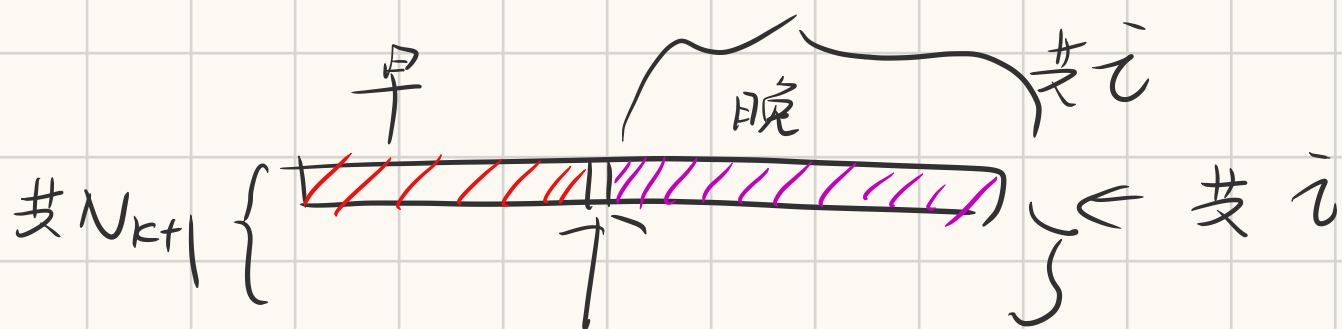
加入的元素, 都在该元素同层或下层.

所以例如第 i 位的元素在第 k 层

当且仅当

$$N_k \leq i \leq N_{k+1}$$

一旦找到 k , 也非常容易找到该元素



同层中前面的一定是更早的元素.

所以 $\boxed{\text{晚}} = N_{k+1} - i$, 此为层内偏移量

所以如何找到 N_k ?

根据上面的观察 (该元素下方的所有物都是新加入的,

所以下方的每一层实际构成了一个 base B 的数)

所以大致可以通过位运算得到最高位.

然而有一点瑕疵，就是 $n < 2B$ ，这实际上会产生一点偏差。我们可以稍微估计一下

[将 x 写成标准 base B 形式， l 为最高非零位

可以证明 $k \geq l-1$,

$$\text{证: } N_{l-1} = \sum_{j=0}^{l-2} n_j B^j \leq 2B \sum_{j=0}^{l-2} B^j = 2B \cdot \frac{B^{l-1}-1}{B-1} \\ \leq 3B^{l-1} \leq B^l \leq x$$

$$\Rightarrow x \geq N_{l-1} \Rightarrow k \geq l-1$$

此时可以再分3种情况讨论:

① $x < N_l, \Rightarrow k = l-1$ (平凡)

② $N_l \leq x < N_{l+1}, \Rightarrow k = l$

③ $N_{l+1} < x$, 此时 x 会落入更高层的某些非空层

此时必然 $k = l'$, 其中 l' 为下一个非空层



如何找到下一个非空层? ($O(1)$ 时间?)



这里使用一个小型拆分对于 n_j ，因为 $n_j < 2B$ 。

我们可以把 n_j 拆成两部分。

$$n_j = n_j^0 + Bn_j^1$$

机器字 $N_0 = (n_{r-1}^0, \dots, n_1^0, n_0^0)_B$

$$N_1 = (n_{r-1}^1, \dots, n_1^1, n_0^1)_B$$

某层 j 非空等价于 $n_j^0 > 0$ 或 $n_j^1 > 0$

找 $N_1 \vee N_2$ 去掉低 r 位后的最低有效位
即可，使用机器指令即可

↓ → 得到 k

于是，我们成功在 $O(1)$ 找到该数